🧪 Load Testing Documentation

Complete guide to performance testing for **REST vs GraphQL comparison research**.

📋 Table of Contents

Overview

Purpose

This load testing suite compares **REST** and **GraphQL** API performance under **identical conditions** to provide scientifically valid research data.

Key Principles

- ✅ **Identical Load Patterns** - Same VUs, duration, ramp-up for both APIs
- ✅ **Same Test Data** - Both APIs query the same database
- ✅ **Fair Comparison** - Industry-standard optimizations for each architecture
- ✅ **Reproducible** - Automated scripts with consistent results

Tools Used

- **k6** - Load testing tool (Grafana Labs)
- **Docker Stats** - Resource monitoring
- **Node.js** - Result analysis scripts

Test Scenarios

Scenario 1: Simple List Query

Purpose: Baseline performance measurement

What it tests:

- Basic facility listing (GET /facilities or `query { facilities }`)
- No complex JOINs or nested data
- Pure retrieval performance

Expected behavior:

- Should be fastest scenario
- Minimal latency
- High throughput

Research significance: Establishes baseline performance for both architectures

---

Scenario 2: List with Relational Data

Purpose: Test N+1 query handling

What it tests:

- User bookings with nested user + facility data
- REST: SQL JOINs vs GraphQL: DataLoader batching
- Multiple related entities per request

REST endpoint: `GET /users/:id/bookings` **GraphQL query:**

```
query {
  userBookings(userId: 3) {
    id
    user { fullName }
    facility { name }
  }
}
```

Research significance: Tests core architectural difference - how each handles related data

---

Scenario 3: Filtered Slot Availability

Purpose: Test parameter handling and filtering

What it tests:

- Date-based filtering
- Query parameter parsing
- Business logic execution (slot generation)

REST endpoint: `GET /facilities/:id/slots?date=2026-01-22` **GraphQL query:**

```
query {
  availableSlots(input: {
    facilityId: 1
    date: "2026-01-22"
  }) {
    slots { startTime endTime isAvailable }
```

```
    }
  }
```

Research significance: Tests how each architecture handles parameterized queries

---

Scenario 4: Nested User Dashboard

Purpose: Test deep data fetching

What it tests:

- Multiple levels of nesting (user → bookings → facility + user)
- Aggregations (count, sum)
- Complex composite responses

REST endpoint: `GET /users/:id/dashboard` **GraphQL query:**

```
query {
  userDashboard(userId: 3) {
    user { fullName }
    bookings {
      facility { name }
      user { fullName }
    }
    bookingCountToday
    totalSpent
  }
}
```

Research significance:

- Tests GraphQL's strength (flexible nesting)
- Tests REST's challenge (composite endpoints or multiple requests)

Scenario 5: Booking Creation (Write Operations)

Purpose: Test write performance and validation

What it tests:

- POST requests vs GraphQL mutations
- Input validation
- Conflict detection
- Database writes

REST endpoint: `POST /bookings` **GraphQL mutation:**

```

```

```
mutation {
  createBooking(input: {
    userId: 3
    facilityId: 1
    bookingDate: "2026-01-22"
    startTime: "14:00:00"
    endTime: "15:00:00"
  }) {
    id status
  }
}
```

Expected behavior:

- Higher latency than reads
- Some conflicts (409 errors) are expected and acceptable
- Lower throughput

Research significance: Tests mutation/write operation performance

---

Scenario 6: Mixed Read-Write Workload

Purpose: Simulate realistic user behavior

Distribution:

- 70% Browsing (list/view facilities)
- 20% Availability checks (slot queries)
- 10% Booking creation

What it tests:

- Real-world usage patterns
- Performance under mixed load
- System behavior with varied request types

Research significance: Most realistic scenario - mimics actual production usage

---

Metrics Collected

Performance Metrics (k6)

Metric	Description	Unit	Threshold
Average Latency	Mean response time	ms	< 200 ms
P50 Latency	Median response time	ms	< 200 ms
P95 Latency	95th percentile	ms	< 500 ms

Metric	Description	Unit	Threshold
P99 Latency	99th percentile	ms	< 1000 ms
Throughput	Requests per second	req/s	> 50
Error Rate	Failed requests	%	< 1%
Data Received	Response payload size	MB	-
Data Sent	Request payload size	MB	-

Resource Metrics (Docker Stats)

Metric	Description	Unit
CPU Usage	CPU utilization	%
Memory Usage	RAM consumption	MB
Memory Limit	Allocated memory	MB
Network Input	Data received	MB
Network Output	Data sent	MB

Custom Metrics (Scenario-Specific)

- **Facility Count** - Number of facilities retrieved
- **Bookings Retrieved** - Number of bookings fetched
- **Available Slots** - Count of available time slots
- **Booked Slots** - Count of unavailable slots
- **Successful Bookings** - Bookings created successfully
- **Booking Conflicts** - Conflict errors encountered

Setup & Installation

Prerequisites

1. **Docker & Docker Compose** (for running APIs)
2. **k6** (load testing tool)
3. **Node.js 20+** (for analysis scripts)

Install k6

Windows:

```
choco install k6
```

macOS:

```
brew install k6
```

Linux:

```
sudo gpg -k
sudo gpg --no-default-keyring --keyring /usr/share/keyrings/k6-archive-
keyring.gpg --keyserver hkp://keyserver.ubuntu.com:80 --recv-keys
C5AD17C747E3415A3642D57D77C6C491D6AC1D69
echo "deb [signed-by=/usr/share/keyrings/k6-archive-keyring.gpg]
https://dl.k6.io/deb stable main" | sudo tee
/etc/apt/sources.list.d/k6.list
sudo apt-get update
sudo apt-get install k6
```

Verify installation:

```
k6 version
```

Setup Test Environment

```
cd aps-backend

# Create test directories
mkdir -p tests/k6/scenarios
mkdir -p tests/k6/config
mkdir -p tests/k6/utils
mkdir -p tests/results/rest
mkdir -p tests/results/graphql
mkdir -p scripts

# Make scripts executable
chmod +x tests/k6/run-all.sh
chmod +x scripts/monitor-resources.sh
chmod +x scripts/compare-results.js

# Start backend services
docker-compose up -d

# Verify services are healthy
docker-compose ps
```

Running Tests

Quick Start (All Scenarios)

```
# Run all 6 scenarios for both REST and GraphQL
./tests/k6/run-all.sh
```

Duration: ~60-90 minutes total

- Each scenario: ~5 minutes
- 6 scenarios × 2 APIs = 12 test runs
- Cool-down periods between tests

Run Individual Scenario

REST:

```
k6 run tests/k6/scenarios/01-simple-list-rest.js
```

GraphQL:

```
k6 run tests/k6/scenarios/01-simple-list-graphql.js
```

Custom Load Configuration

Edit `tests/k6/config/rest.config.js` or `graphql.config.js`:

```
export const LOAD_CONFIGS = {
  read: {
    stages: [
      { duration: '30s', target: 10 },   // Ramp to 10 VUs
      { duration: '1m', target: 25 },    // Increase to 25
      { duration: '2m', target: 50 },    // Sustained at 50
      { duration: '1m', target: 100 },   // Peak at 100
      { duration: '30s', target: 0 },    // Ramp down
    ],
  },
};
```

Monitor Resources During Testing

```
# Monitor for 5 minutes, output to CSV
./scripts/monitor-resources.sh 300 results/resources.csv all

# Monitor specific container
./scripts/monitor-resources.sh 300 results/rest-resources.csv aps-rest-api
```

Analyzing Results

Compare REST vs GraphQL

```
node scripts/compare-results.js
```

Output example:

```
================================================================================
=====
Scenario: Simple List Query
================================================================================
=====

📊 Performance Metrics:
--------------------------------------------------------------------------------
-----
Metric                    REST           GraphQL        Difference
--------------------------------------------------------------------------------
-----
Avg Latency (ms)          45.23          48.91          +8.14% ✗
P50 Latency (ms)          42.10          46.33          +10.05% ✗
P95 Latency (ms)          89.45          102.78         +14.90% ✗
P99 Latency (ms)          156.23         178.90         +14.51% ✗
--------------------------------------------------------------------------------
-----
Requests/sec              523.45         498.23         -4.82% ✓
Total Requests            157035         149468
--------------------------------------------------------------------------------
-----
Error Rate (%)            0.12           0.09           -25.00% ✓
--------------------------------------------------------------------------------
-----
Data Received (MB)        45.23          52.78          +16.69% ✗
Data Sent (MB)            2.34           3.12           +33.33% ✗
--------------------------------------------------------------------------------
-----

🏆 Analysis:
   • REST is 14.9% faster (p95 latency)
   • REST handles 5.1% more requests/sec
   • GraphQL transfers 17.8% more data
```

View Raw Results

k6 Summary:

```
cat tests/results/rest/01-simple-list-summary.json
```

Resource Usage:

```
cat tests/results/rest/01-simple-list-resources.csv
```

Export for Thesis

All results are in JSON/CSV format for easy import into:

- Excel/Google Sheets
- Python (pandas)
- R (ggplot2)
- LaTeX tables

Troubleshooting

Issue: k6 not found

Solution:

```
# Verify installation
k6 version

# Reinstall if needed
choco install k6  # Windows
brew install k6    # macOS
```

Issue: Services not healthy

Solution:

```
# Check service status
docker-compose ps

# View logs
docker-compose logs rest-api
docker-compose logs graphql-api

# Restart if needed
docker-compose restart
```

Issue: High error rates

Possible causes:

1. Database connection issues
2. Too many concurrent users
3. Conflicts in booking creation (expected for Scenario 5)

Solution:

```
# Check database
docker-compose logs postgres

# Reduce load in config files
# Edit LOAD_CONFIGS.read.stages in config files
```

Issue: Inconsistent results

Solution:

- Ensure no other processes are running
- Run tests during low system load
- Increase cool-down periods between tests
- Run multiple iterations and average results

Research Methodology

Experimental Controls

To ensure **scientifically valid comparison**:

1. **Identical Hardware**
    - Same Docker host
    - Same resource limits (1 CPU, 512MB RAM per container)
    - Same network configuration
2. **Identical Load Patterns**
    - Same VU counts
    - Same test duration
    - Same ramp-up/ramp-down
3. **Identical Data**
    - Same database
    - Same seed data
    - Same test users
4. **Identical Business Logic**
    - Both APIs use same service layer

- Same validation rules
- Same conflict detection

Variables

Independent Variable:

- API architecture (REST vs GraphQL)

Dependent Variables:

- Response time (latency)
- Throughput (requests/sec)
- Error rate
- CPU usage
- Memory usage
- Network I/O

Controlled Variables:

- Hardware resources
- Database
- Business logic
- Load patterns
- Test data

Statistical Validity

- **Sample Size:** 5-minute sustained load per scenario
- **Repetition:** Run each scenario 3 times, report median
- **Outlier Handling:** P95/P99 percentiles account for outliers
- **Error Margin:** Report standard deviation

Reporting Results

For your thesis (Chapter 4 - Results):

Table IV: Performance Metrics Comparison

```
Scenario              | Metric        | REST    | GraphQL | Difference
----------------------|---------------|---------|---------|------------
Simple List           | P95 Latency   | 89.45ms | 102.78ms| +14.90%
                      | Throughput    | 523 rps | 498 rps | -4.82%
...
```

Table V: Resource Utilization Comparison

```
Scenario              | Metric        | REST    | GraphQL | Difference
----------------------|---------------|---------|---------|------------
```

```
Simple List          | Avg CPU       | 45.2%   | 52.1%   | +15.26%
                     | Avg Memory    | 234 MB  | 256 MB  | +9.40%
...
```

File Locations

Test Scripts

- `tests/k6/scenarios/` - All 6 scenarios × 2 APIs = 12 test files
- `tests/k6/config/` - Load configurations
- `tests/k6/utils/` - Helper functions (auth, data generation)

Results

- `tests/results/rest/` - REST API results (JSON + CSV)
- `tests/results/graphql/` - GraphQL API results (JSON + CSV)

Scripts

- `tests/k6/run-all.sh` - Run all tests
- `scripts/monitor-resources.sh` - Resource monitoring
- `scripts/compare-results.js` - Result analysis

Next Steps

After running all tests:

1. **Analyze Results**

   ```
   node scripts/compare-results.js
   ```

2. **Generate Charts**
   - Import JSON files into Excel/Python/R
   - Create latency comparison charts
   - Create throughput bar charts
   - Create resource usage line charts

3. **Statistical Analysis**
   - Calculate mean, median, std dev
   - Perform t-tests for significance
   - Create confidence intervals

4. **Write Chapter 4 (Results)**
   - Report all metrics in tables

    - Include charts/figures
    - Discuss findings
    - Analyze trade-offs

References

- k6 Documentation: https://k6.io/docs/
- Docker Stats: https://docs.docker.com/engine/reference/commandline/stats/
- Research Methodology: See thesis Chapter 3

Questions? Check troubleshooting section or review test scripts for implementation details.